Phase 1 - Week 1 - Lesson Day 3

NumPy Operations

Today's Objectives

Yesterday you learned how to create, index, and reshape arrays. Today is about actually **working** with the numbers inside them - doing math across whole arrays at once, summarizing data, and filtering it based on conditions. These four skills are used in almost every single data science task you'll ever do.

- Understand vectorized operations and why loops are avoided in NumPy
- Understand broadcasting between arrays of different shapes
- Use aggregate functions to summarize data (sum, mean, std, min, max)
- Use boolean masking to filter arrays based on a condition

1. Vectorized Operations

Definition: *Vectorization* means applying an operation to an entire array at once, instead of looping through it element by element in Python. NumPy performs the loop internally using fast, low-level compiled code, which is why it is dramatically faster than a plain Python for-loop over a list.

The slow way, using a Python loop:

```
arr = np.array([1, 2, 3, 4])

doubled = []
for x in arr:
    doubled.append(x * 2)
print(doubled)   # [2, 4, 6, 8]
```

The fast, vectorized way - identical result, one line:

```
doubled = arr * 2
print(doubled)   # [2, 4, 6, 8]
```

This pattern works for every basic operation: **arr + 5**, **arr - 2**, **arr * 3**, **arr / 2**, **arr ** 2** - each one is automatically applied to every single element. **Rule of thumb going forward:** if you catch yourself writing a for-loop over a NumPy array to do simple math, there is almost always a faster vectorized way to write it instead.

2. Broadcasting

Definition: *Broadcasting* is NumPy's rule system for performing operations between arrays of different shapes, by automatically "stretching" the smaller array to match the larger one's shape - without actually copying any data in memory.

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
row = np.array([10, 20, 30])

result = matrix + row
print(result)
# row is applied to every row of the matrix:
# [[11, 22, 33],
#  [14, 25, 36]]
```

Broadcasting only works when the shapes are compatible - here, the row has 3 elements, matching the matrix's 3 columns, so NumPy "broadcasts" it down each row. If the shapes don't line up (for example, a row of 4 elements added to a matrix with 3 columns), NumPy raises an error.

3. Aggregate Functions

Definition: *Aggregate functions* take an entire array and reduce it down to a single summary value (or one value per row/column when you specify an axis). These are the exact same concepts as Excel's SUM, AVERAGE, MIN, and MAX functions - just applied to NumPy arrays.

```
arr = np.array([4, 8, 15, 16, 23, 42])

print(arr.sum())    # 108   -> total of all values
print(arr.mean())   # 18.0  -> average value
print(arr.std())    # standard deviation - how spread out the values are
print(arr.min())    # 4     -> smallest value
print(arr.max())    # 42    -> largest value
```

For 2D arrays, you can control the **direction** of the aggregation using the **axis** parameter: **axis=0** aggregates down each column, **axis=1** aggregates across each row.

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])

print(matrix.sum(axis=0))   # sum down each column -> [5, 7, 9]
print(matrix.sum(axis=1))   # sum across each row  -> [6, 15]
```

A simple way to remember it: **axis=0** moves *down* through the rows (collapsing them into one row of column totals), and **axis=1** moves *across* through the columns (collapsing them into one column of row totals).

4. Boolean Masking

Definition: A *boolean mask* is an array of True/False values produced by applying a condition to every element of an array. You can then use that mask to filter the original array, keeping only the elements where the mask is True. This is the NumPy equivalent of filtering rows in Excel or SQL with a WHERE clause.

```
arr = np.array([10, 15, 20, 25, 30])

mask = arr > 18
print(mask)        # [False, False, True, True, True]

print(arr[mask])   # [20, 25, 30] -> only the values where mask is True
```

In practice, this is almost always written in one line, without a separate mask variable:

```
print(arr[arr > 18])   # exact same result as above
```

You can also combine multiple conditions using **&** (and) and **|** (or) - each condition must be wrapped in its own parentheses.

```
print(arr[(arr > 12) & (arr < 28)])   # values greater than 12 AND less than 28
```

Today's Exercise

Open the accompanying notebook file **day-3-numpy-operations.ipynb** and complete the exercises below inside it. No loops allowed anywhere in this exercise - every answer should use vectorized operations, aggregate functions, or boolean masking.

Part A - Vectorized Operations & Broadcasting (3 questions)

1. Apply a discount

Given prices = [100, 250, 75, 300, 50], apply a 20% discount to every price in one vectorized line (no loop).

2. Temperature conversion

Given a NumPy array of Celsius temperatures [0, 15, 30, 45], convert all of them to Fahrenheit using the formula (C * 9/5) + 32, in one line.

3. Broadcast a bonus

Given a 2D array of monthly sales for 3 employees over 2 months (shape 3x2), add a flat bonus of 500 to every value using broadcasting.

Part B - Aggregate Functions (3 questions)

4. Sales summary

Given a sales array of 10 numbers, calculate and print the sum, mean, min, and max in one go.

5. Row and column totals

Given a 2D array representing sales for 3 products over 4 weeks (shape 3x4), calculate the total per product (row) and the total per week (column).

6. Standard deviation comparison

Given two arrays of exam scores, calculate the standard deviation of each and state in a comment which group of scores is more spread out.

Part C - Boolean Masking (4 questions)

7. Above average filter

Given a sales array, calculate its mean, then use boolean masking to print only the values above the mean (no loop).

8. Passing scores

Given an array of exam scores, use boolean masking to print only the scores greater than or equal to 60 (a 'pass' threshold).

9. Combined condition

Given an array of ages, use boolean masking with a combined condition to print only the ages between 18 and 30 (inclusive).

10. Count matching elements

Given an array of temperatures, count how many values are above 25 degrees using masking combined with .sum() (hint: True counts as 1, False as 0).

How to Approach It

If you find yourself reaching for a **for** loop anywhere in this exercise, stop and ask: "can this be written as arr + something, arr[condition], or arr.sum()/.mean() instead?" That instinct - reaching for vectorized operations before loops - is one of the most important habits to build early in data science, since it directly affects how fast your code runs on real datasets.

Once you and Halimo have both completed the exercises in the notebook, bring your answers back to the lesson chat and we'll review them together before moving on to Lesson Day 4 (NumPy Applied).